

Design Document: CliFactory

Terminology & Formulae

- **game_state**: Central dictionary holding all simulation data (capital, inventories, R&D tiers, cooldowns, multipliers, risk flags).
- **grace period (guaranteed months)**: Hazard-free months based on difficulty (Easy = 4, Normal = 2, Hard = 0).

risk percentage: Chance of hazards or negative outcomes. Always between 5% and 100%.

- Formula:

$\text{Risk} = \max(0.05, \min(1.0, \text{BaseChance} - (\text{SafetyTier} \times 0.04)))$

futures penalty: Deduction if monthly futures quota (40 units) is missed.

- Formula:

$\text{Penalty} = \text{TotalCapital} \times 0.10$

marketing bonus: Capital bonus if quotas are met.

- Formula:

$\text{Bonus} = \text{TotalCapital} \times \text{TierBonus}$

TierBonus = {0.05, 0.10, 0.15, 0.25} for Tiers 1–4.

extraction yield: Passive raw material generation.

- Formula:

$\text{Yield} = 25 + (\text{ExpansionTier} \times 5) + \text{EfficiencyBonus}$

EfficiencyBonus: Tier 3 = +20, Tier 4 = +35.

refining capacity: Passive conversion of raw materials to goods.

- Formula:

$\text{Capacity} = 18 + (\text{ExpansionTier} \times 15) + \text{EfficiencyBonus}$

EfficiencyBonus: Tier 1 = +5, Tier 2 = +10, Tier 3 = +5.

- **monopoly victory**: Win condition. Requires difficulty-specific thresholds for capital, output, maxed R&D, and fulfilled futures.

Program Flow

1. Prompt user for difficulty.
2. Create initial game_state (capital + grace period).
3. Start loop:
 - If capital ≤ 0 → Bankruptcy screen → Exit.
 - If victory achieved → Corporate Hegemony screen → Exit.
4. Render Dashboard (capital, inventories, upkeep, flags, quotas).
5. Prompt for menu choice (1–9).
6. **Major Actions (1–5)**: Check slot availability → prompt sub-choice → update state → lock slot.
7. **Minor Actions (6–7)**: Check availability (<2 used) → show risk → resolve outcome → update state.
8. **End Month (8)**:
 - Check cave-in hazard.
 - Apply rollover (extraction, refining, maintenance, futures).
 - Print action log.
 - 50% chance of Monthly Event → handle binary choice (strict "1"/"2").
 - Increment month → return to step 3.
9. **Exit (9)**: Print exit sequence → End.
10. **Invalid choice**: Print error → wait → return to step 3.

File Organization

- **factory_logic.py**: Functional core. Pure functions only. No I/O. Designed for unit testing.
- **main.py**: Imperative shell. Handles I/O, execution loop, UI, and routing.

Data Structures

- **Dictionary (dict)**: Holds game_state. Keys include:
 - Core Economy: capital, raw_materials, finished_goods, costs, limits.
 - Progression: difficulty, month, grace period, expansion tier, R&D levels.
 - Turn Constraints: major/minor action flags, skips, locks.
 - Modifiers & Risk: event chance, multipliers, futures, bonuses, sales tracking.
- **List (list)**: Used for logs, tier names, and monthly event pools.
- **Strings (str)**: Strict input validation. Only numeric strings are accepted ("1", "2", "3"). Letters rejected.

Function Signatures

factory_logic.py (functional core)

Function Header	Purpose
<pre>def create_initial_game_state(difficulty_name: str, starting_capital: float, guaranteed_months: int) -> dict:</pre>	Initializes and returns the baseline state dictionary.
<pre>def is_bankrupt(game_state: dict) -> bool:</pre>	Checks if capital ≤ 0 .
<pre>def has_achieved_monopoly(game_state: dict) -> bool:</pre>	Evaluates difficulty-specific victory conditions.
<pre>def get_current_factory_title(game_state: dict) -> str:</pre>	Maps expansion tier integer to its title string.
<pre>def calculate_effective_risk(game_state: dict) -> float:</pre>	Computes risk, strictly bounded between 0.05 and 1.0.
<pre>def evaluate_minor_action_risk(game_state: dict) -> bool:</pre>	Rolls RNG against effective risk; returns True if hazard triggers.

<pre>def execute_factory_expansion(game_state: dict, strategy_type: str) -> dict:</pre>	Deducts costs, increments tier, applies strategy modifiers.
<pre>def execute_rd_investment(game_state: dict, selected_field: str) -> dict:</pre>	Increments R&D tier and applies static modifiers.
<pre>def execute_sales_contract(game_state: dict, quantity_to_sell: int) -> tuple[dict, float, int]:</pre>	Processes goods sold against limits and multipliers.
<pre>def roll_monthly_event(state: dict) -> tuple[bool, None, str]:</pre>	Randomly selects and returns monthly event outcomes.
<pre>def process_end_of_month_rollover(game_state: dict, cave_in_occurred: bool) -> tuple[dict, list]:</pre>	Applies monthly rollover logic and returns updated state + logs.

main.py (imperative shell)

Function Header	Purpose
<pre>def clear_screen() -> None:</pre>	Clears the terminal output safely across OS platforms.
<pre>def select_difficulty() -> tuple[str, float, int]:</pre>	Prompts user for numeric choice (1–3) and returns starting parameters.

<code>def get_binary_choice() -> str:</code>	Strict validation loop forcing user to input exactly "1" or "2".
<code>def handle_monthly_event(state: dict) -> dict:</code>	Renders event text, routes binary choice, mutates state.
<code>def run_game() -> None:</code>	Owns the main loop: UI routing, state holding, and overall execution.

Function-Level Algorithms

factory_logic.py

- `create_initial_game_state`
 - Populate dictionary with base variables (capital, inventories, R&D levels, factory tiers, modifiers, constraints).
 - Use difficulty parameters to set starting values.
 - Return initialized state.
- `is_bankrupt`
 - If `game_state["capital"] <= 0`: return True.
 - Else: return False.
- `has_achieved_monopoly`
 - Check difficulty.
 - Compare current capital, maxed R&D tiers, output rate, and futures fulfilled against thresholds.
 - Return True if all conditions met, else False.
- `get_current_factory_title`
 - Use `factory_expansion_tier` as index.
 - Return corresponding string from `factory_tier_names`.
- `calculate_effective_risk`
 - Fetch `negative_event_chance`.
 - Subtract (`SafetyTier × 0.04`).
 - Clamp result between 0.05 and 1.0.
 - Return risk value.
- `evaluate_minor_action_risk`
 - Call `calculate_effective_risk`.
 - Generate random float.
 - If float < risk: return True (hazard).
 - Else: return False.

- `execute_factory_expansion`
 - Copy state.
 - Deduct expansion cost from capital.
 - Increment expansion tier.
 - Lock major action.
 - Apply strategy-specific upkeep and risk modifiers.
 - Return updated state.
- `execute_rd_investment`
 - Copy state.
 - Deduct 100 capital.
 - Increment selected R&D field.
 - Apply modifiers (maintenance, sell limit, risk).
 - If `current_month <= guaranteed_months`: no fail chance.
 - Return updated state.
- `execute_sales_contract`
 - Calculate unit price (logistics tier multipliers, temporary bonuses, futures multiplier).
 - Deduct sold goods from inventory.
 - Add revenue to capital.
 - Return updated state, revenue, units sold.
- `roll_monthly_event`
 - 50% chance: return None, None.
 - Else: calculate negative chance (min floor 0.10).
 - Randomly select event string (positive or negative).
 - Return event outcome.
- `process_end_of_month_rollover`
 - Copy state.
 - Evaluate futures quota (apply penalty or track success).
 - Check marketing quotas (apply tier bonuses).
 - Calculate mining yield (base + tier + efficiency bonus).
 - If cave-in: yield = 0, deduct 90 capital.
 - Calculate refining capacity (base + tier + efficiency bonus).
 - Convert raw materials to goods.
 - Deduct maintenance cost.
 - Reset turn constraints.
 - Return updated state and action logs.

main.py

clear_screen

- Detect operating system.
- If Windows → run `os.system("cls")`.
- Else → run `os.system("clear")`.
- Ensures clean terminal output across platforms.

select_difficulty

- Start while True loop.
- Prompt user for input ("1", "2", or "3").
- If valid:
 - Map choice to difficulty name, starting capital, and grace period.
 - Return tuple (difficulty_name, starting_capital, guaranteed_months).
- Else:
 - Print error message.
 - Loop again until valid input is given.

get_binary_choice

- Start while True loop.
- Prompt user for input.
- If input is exactly "1" or "2" → return it.
- Else:
 - Print [!] Invalid choice. Please enter 1 or 2.
 - Loop again.
- Guarantees no letters or invalid strings pass validation.

handle_monthly_event

- Call roll_monthly_event(state).
- If no event triggered → return unchanged state.
- Else:
 - Parse event string (positive or negative).
 - If negative:
 - Prompt user for binary choice using get_binary_choice.
 - Apply logic based on choice.
 - Mutate state accordingly.
- Return updated state.

run_game

- Call select_difficulty() → initialize game_state.
- Start while True loop:
 - Check bankruptcy (is_bankrupt). If True → render Bankruptcy screen → exit.
 - Check monopoly victory (has_achieved_monopoly). If True → render Victory screen → exit.
 - Render dashboard (capital, inventories, upkeep, flags, quotas).
 - Prompt user for numeric input (1–9).
 - Route input via if/elif:

- Major actions (1–5): Check if slot available → execute expansion/R&D → lock slot.
- Minor actions (6–7): Check usage count → calculate risk → resolve outcome.
- End month (8): Evaluate cave-in → call rollover → print logs → trigger `handle_monthly_event`.
- Exit (9): Print exit sequence → break loop.
- Invalid input: Print error → wait → loop back.
- Loop continues until bankruptcy or victory.